EDMS Python & Database Study Guide

A field guide to the Python, SQLAlchemy, FastAPI, and security concepts introduced while building EDMS.

Use this as a reading companion, not a memorization test. Trace each example back to the source, rewrite it by hand, and predict what it returns before running it.

> **Learning target:** understand the mechanism behind the syntax well enough to rebuild the feature without copying it. The goal is a stronger mental model, not faster typing today.

1. The layers: business objects and database objects

A relational database stores rows in tables. Python works with objects. An ORM (object-relational mapper) maps between those worlds and tracks database objects through a session.

Kind	EDMS example	Responsibility
Domain object	`PurchaseOrder`, `PurchaseLine`	Business meaning and behavior: approve an order, receive goods, calculate a line total, reject duplicates.
ORM record	`PurchaseOrderRecord`, `PurchaseOrderLineRecord`	Table/column mapping, keys, constraints, and relationships understood by SQLAlchemy.
Repository	`PurchaseOrderRepository`	Queries and persistence; translates a domain order to database records when saving.
Session	`Session`	Unit of work: tracks loaded/new records and sends SQL within a transaction.

```
PurchaseOrder domain object
    -- map fields --> PurchaseOrderRecord + PurchaseOrderLineRecord
    -- SQLAlchemy Session --> database rows
```

A dataclass does not become persistent just because it is an object. Changing a plain `PurchaseOrder` changes that in-memory object. A mapped ORM object is associated with SQLAlchemy's session; SQLAlchemy can notice changes and persist them when the transaction is flushed/committed.

This project deliberately has separate domain and ORM classes. That protects business rules from database details, but means a repository must map between the two. Current read methods return ORM records: appropriate for a persistence-focused read path, but callers should know they are receiving ORM objects.

2. Tables, keys, and relationships

Think of each table as one kind of fact. A purchase order is the header; each purchase line is a separate row because one order can contain many lines.

```
purchase_orders
  id (primary key)
  supplier_name snapshot
  status
  total

purchase_order_lines
  id (primary key)
  purchase_order_id (foreign key -> purchase_orders.id)
  product_id
  product_name snapshot
  quantity, received_quantity
  manufacturer_name snapshot
  unit_price snapshot
```

- **Primary key:** identifies one row, such as `purchase_orders.id`.
- **Foreign key:** requires a referenced row to exist. A line's `purchase_order_id` points to its owning order.
- **One-to-many:** one order has many lines; each line belongs to one order.
- **Relationship:** SQLAlchemy's Python-side navigation between mapped classes. It does not replace the foreign-key column; both describe different sides of the mapping.

`lines = relationship(...)` on the order record lets Python navigate `order.lines`. `purchase_order = relationship(...)` on a line lets Python navigate back. `back_populates` names the corresponding attribute on the other mapped class so SQLAlchemy keeps both sides aligned.

3. Why values are saved as snapshots

A purchase line records the terms agreed at a moment in time. If the catalog price or supplier details change later, old purchase orders should not silently change their historical meaning.

- `unit_price` is the agreed unit price for this order line, not necessarily today's catalog price.
- `product_name`, `manufacturer_name`, and `supplier_name` are saved copies as they were when the order was made.
- `product_id` is the identity used to find the catalog product. A future schema may make it a foreign key; the copied name still serves as historical display text.

The repository's save method constructs new ORM records using values from the domain objects. Those values are copied into the pending database records. Mutating the original domain objects after saving does not rewrite those snapshots automatically.

> **Design question:** decide which values mean “current master data” and which mean “what was true/agreed at transaction time.” IDs usually connect to master data; commercial terms and historical display values are often snapshotted.

4. Database constraints: the last line of defense

Python validation gives useful, immediate errors to normal application callers. Database constraints protect stored data even if another code path forgets that validation.

- `CheckConstraint("quantity > 0")`: quantity must be positive.
- `CheckConstraint("received_quantity >= 0 AND received_quantity <= quantity")`: received quantity must stay within the ordered amount.
- `CheckConstraint("unit_price >= 0")`: a price cannot be negative.
- `UniqueConstraint("purchase_order_id", "product_id")`: one product identity can occur at most once within one order.

A unique constraint across two columns means their *pair* must be unique. The same product may appear on another order; the same order may contain other products. Domain validation in `PurchaseOrder.__post_init__` gives a friendly error before persistence, while the database constraint handles races or alternate writers.

The duplicate rule assumes a product ID represents the exact thing that must not repeat. If two configurations of one base product may be ordered separately, the key should identify the configuration instead.

Schema caution: SQLAlchemy's `Base.metadata.create_all()` creates missing tables; it does not modify an existing table to add new constraints. Existing databases need a migration, and existing duplicate/invalid rows must be corrected before the constraint can be applied.

5. Indexes and practical query cost

An index is an extra data structure the database maintains to find rows by selected columns. It speeds matching reads but consumes storage and adds work to inserts/updates. Add indexes to serve known queries, not automatically to every column.

The old lookup searched Python lists of orders and their lines. That meant visiting many objects and nested iteration. The current design asks the database for the line matching both its owning order and its product ID. The unique constraint on those columns also creates an index in common relational databases.

Correction to the earlier shorthand: indexed database lookup is not generally O(1). A typical B-tree lookup is approximately O(log n), plus SQL execution and I/O costs. The important win here is that the application does not fetch and scan all orders and lines itself.

6. Reading SQLAlchemy's select syntax

SQLAlchemy 2.x builds a statement using Python expressions. Method chaining successively adds clauses to one statement.

```
statement = select(PurchaseOrderLineRecord).where(
    PurchaseOrderLineRecord.purchase_order_id == purchase_order_id,
    PurchaseOrderLineRecord.product_id == product_id,
)
line = session.scalar(statement)
```

Expression	Meaning	SQL idea
`select(Model)`	Build a statement selecting mapped instances of that model.	`SELECT ... FROM ...`
`.where(condition, condition)`	Filter rows; multiple conditions are combined with AND.	`WHERE a = ? AND b = ?`
`.order_by(column)`	Sort returned rows. It is `order_by`, not `order`.	`ORDER BY column`
`.limit(5)`	Return no more than the requested number of rows.	`LIMIT 5`

Building `statement` is like composing a query plan description. It usually does not contact the database until a session executes it. The `==` in the expression is overloaded by SQLAlchemy: it constructs a SQL comparison rather than immediately comparing two ordinary Python values.

7. What scalar() and scalars() return

Both methods execute the statement through the session. Their difference is what they expose from the result:

- `session.scalar(statement)` returns the first selected value from the first row, or `None` when there is no row. Use it when expecting one value/record, such as a row selected by a primary key or unique key.
- `session.scalars(statement)` returns a result you can iterate over, containing the selected value from each row. Use it for zero, one, or many matching records.
- `session.execute(statement)` exposes full rows/result objects and is useful when selecting several columns or expressions.

```
# One matching ORM object, or None
line = session.scalar(select(PurchaseOrderLineRecord).where(...))

# Many ORM objects
lines = session.scalars(select(PurchaseOrderLineRecord).where(...))
for line in lines:
    print(line.product_name)
```

Because the database constraint guarantees at most one matching line for an order/product pair, `scalar()` fits the current purchase-line query. Without uniqueness, it could hide the fact that multiple rows matched; use `scalars()` and inspect the collection if multiplicity matters.

8. What relationship loading means

ORM relationships may load related rows later, immediately, or in a batched follow-up query. The mapping currently says `lazy="selectin"` for order lines. When orders are loaded, SQLAlchemy fetches their lines with a second query using the set of order IDs. This avoids one line-query per order (the N+1 query problem).

The per-query option `selectinload(PurchaseOrderRecord.lines)` can request the same strategy for one query. Since this mapping already sets it as the default, repeating the option there is redundant. Query options are useful when a particular query should load relationships differently from the mapping default.

9. Why functions appear in class declarations

Python class bodies execute when the class is defined. A line such as `lines = relationship(...)` calls

the function `relationship` immediately. SQLAlchemy returns a special object (a descriptor) and stores that object as the class attribute. The descriptor knows how to provide relationship behavior on instances.

```
lines: Mapped[list["PurchaseOrderLineRecord"]] = relationship(
    back_populates="purchase_order",
    cascade="all, delete-orphan",
    lazy="selectin",
)
```

This does **not** mean each order's `lines` attribute is a function. It means the class declaration calls SQLAlchemy's mapping API to configure the attribute.

Call now versus pass for later

```
column = mapped_column(String(200))  # calls mapped_column now
callback = some_function             # stores/passes the function itself
result = some_function()             # calls the function now
```

Frameworks sometimes accept callables so they can invoke them later. For example, a dataclass `default_factory=list` passes the callable `list`; the dataclass calls it separately for each instance. By contrast, `default=list()` calls it now and passes one already-created list. Avoid a shared mutable default.

In this mapping, `relationship(...)` and `mapped_column(...)` are called in the class body; they return SQLAlchemy mapping objects. If an API accepts a function as a callback/default factory, look for whether the code passes `function` or calls `function()`.

10. Python syntax used in the project

Syntax	How to read it
`Mapped[int]`	A type annotation saying SQLAlchemy maps this attribute as an integer column.
`str | None`	Either a string or `None`; modern union type syntax.
`list[PurchaseLine]`	A list whose items should be `PurchaseLine` instances.
`field(default=..., init=False)`	Dataclass field metadata; omit this field from the generated constructor and use the specified default.
`@property`	Lets a method be read like an attribute, e.g. `order.total`.

`self._status = ...`	Leading underscore conventionally marks an implementation detail; it is not strict privacy.
`[expr for item in items]`	List comprehension: build a list by transforming each item.
`set(values)`	Unique collection; comparing list and set lengths can detect duplicates.
`Enum` and `.value`	Use named states in Python, store a stable primitive string such as `"draft"` in the database.

In the duplicate check, the list comprehension extracts every product ID. A set removes duplicates. If both lengths differ, at least one ID appeared more than once.

11. Sessions and transactions

A SQLAlchemy `Session` is a unit of work, not the database itself. It tracks objects, stages inserts/updates/deletes, executes queries, and coordinates a transaction.

- `session.add(record)` stages a new ORM object for insertion.
- `session.flush()` sends pending SQL inside the current transaction. It can obtain generated IDs, but it does not commit.
- `commit` makes the transaction durable; `rollback` undoes its uncommitted changes.

The database helper uses `sessionmaker.begin()` as a context manager: normal exit commits; an exception rolls back and is re-raised. Saving an order and its lines in one transaction means they succeed or fail together.

`@contextmanager` comes from Python's standard library. It turns a generator function with one `yield` into a `with`-compatible manager. The code before `yield` runs on entry; code after it runs on exit. Here SQLAlchemy already owns commit/rollback, and the wrapper provides a convenient session-scope API.

12. A practical learning loop

1. Read one small method and write down its inputs, output, and side effects.
2. Rewrite a tiny version without SQLAlchemy. For example, filter a Python list by order ID and product ID.

3. Translate the list operation into a `select(...).where(...)` statement.
4. Predict the return type before calling `scalar()` or `scalars()`.
5. Try invalid data and observe whether domain validation or the database constraint rejects it.
6. Change one value after saving and query again to see which values were snapshots.
7. Draw the tables, keys, and relationship arrows from memory.

> **Manual recall prompt:** Explain, without looking: why one order maps to many line rows; what the unique pair means; why a unit price is snapshotted; what `select()` constructs; when the query executes; and why `scalar()` fits one row.

13. Small exercises

1. Write a plain Python loop that finds all lines for one order. Then express the same condition in SQLAlchemy `.where()`.
2. Remove the product ID from a line record in a scratch database. What does a required column prevent?
3. Attempt to insert two lines with the same order ID and product ID. Identify which layer rejects it first.
4. Change a product name and price after saving. Which database values should remain unchanged, and why?
5. Make a statement select only `PurchaseOrderRecord.id`. Compare `scalar()` with `scalars()`.
6. Remove `lazy="selectin"` in a scratch experiment and inspect how loading multiple orders affects query count.

14. Authentication versus authorization

Authentication answers *"who is making this request?"* Authorization answers *"may this identity perform this action?"* EDMS keeps them separate. Logging in proves identity; every protected route then checks a permission.

Concern	EDMS mechanism	Failure code
Who are you?	Username + password, then a bearer token	401 Unauthorized

Are you allowed?	Role-derived permission check	403 Forbidden
Are you still active?	`is_active` check on login and on every token lookup	403 Forbidden

The role is stored, but the permission set is **derived** from it:

```
class Role(Enum):
    CEO = "ceo"
    ADMINISTRATOR = "administrator"
    SALES_OFFICER = "sales_officer"
    # ...

ROLE_PERMISSIONS: dict[Role, frozenset[str]] = {
    Role.SALES_OFFICER: frozenset({
        CATALOG_READ, INVENTORY_READ, SALES_READ, SALES_WRITE,
    }),
    # ...
}

class User:
    @property
    def permissions(self) -> frozenset[str]:
        return ROLE_PERMISSIONS[self.role]

    def has_permission(self, permission: str) -> bool:
        return permission in self.permissions
```

- **Derived permissions** mean changing a role's rules updates every user at once — no data migration. The trade-off: an old action cannot be audited by asking "what permissions did this user have then?" If that is required, snapshot the role on the audit record.
- **`frozenset`** is immutable and gives average O(1) membership tests. A mutable `set` stored on a shared module constant could be edited by accident at runtime.
- **`@property`** makes `user.permissions` read like data while it is computed on access. There is no `permissions` column in the database.
- **Strings as permission names** ("catalog:write") are stable identifiers at the boundary. Typos are runtime failures, so keeping them as module constants is safer than retyping literals.

> **Mental model:** authentication produces a *principal*; authorization consumes a *(principal, permission)* pair. The API dependency `require_permission("sales:write")` is the consumer; nothing in the domain layer knows HTTP exists.

15. Password hashing: salts, iterations, and constant-time comparison

Passwords are **hashed**, never encrypted. Encryption is reversible with a key; hashing is one-way. If the database leaks, the attacker should not be able to recover passwords or replay the stored value directly.

```
ALGORITHM = "pbkdf2_sha256"
ITERATIONS = 260_000
SALT_BYTES = 16

def hash_password(password: str) -> str:
    salt = secrets.token_bytes(SALT_BYTES)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, ITERATIONS)
    return f"{ALGORITHM}${ITERATIONS}${salt.hex()}${digest.hex()}"
```

Pieces	Why it exists
Salt (16 random bytes)	Two users with the same password get different hashes. Defeats precomputed rainbow tables.
Iteration count (260,000)	Makes each guess expensive. A fast hash like plain SHA-256 is too cheap for passwords.
Stored algorithm name	Allows a future migration to a different scheme while old hashes still verify.
`password.encode("utf-8")`	Hashing functions operate on bytes, not `str`.

```
def verify_password(password: str, stored_hash: str) -> bool:
    algorithm, iterations, salt_hex, digest_hex = stored_hash.split("$")
    if algorithm != ALGORITHM:
        return False
    candidate = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), bytes.fromhex(salt_hex), int(iterations)
    )
    return hmac.compare_digest(candidate, bytes.fromhex(digest_hex))
```

- **`compare_digest`** compares in constant time. A normal `==` can exit on the first differing byte, leaking timing information about how much of the hash was guessed correctly.
- **Recompute, do not decrypt.** Verification hashes the supplied password with the stored salt and iterations, then compares digests.
- **Defensive parsing.** The stored string `algorithm$iterations$salt$digest` is split and validated; malformed or unknown formats return `False` instead of raising.
- **Standard library only.** PBKDF2 ships with Python, so the project needs no crypto dependency. Production systems often use Argon2id or bcrypt/scrypt, which are memory-hard.

16. Bearer tokens: opaque secrets stored as digests

After login the server issues an opaque random token. The client sends it in the `Authorization` header; the server looks up its digest.

```
def generate_token() -> str:
    return secrets.token_urlsafe(32)      # ~256 bits of entropy

def digest_token(token: str) -> str:
    return hashlib.sha256(token.encode("utf-8")).hexdigest()
```

Choice	Reason
Random opaque token, not a JWT	No signing library, no parsing pitfalls, and revocation is a single row update.
Store `sha256(token)`, not the token	A database leak does not hand over usable sessions. The raw token exists only in the client's memory.
Fast hash is acceptable here	The token already has ~256 bits of entropy, so brute-force guessing is infeasible. Passwords need slow hashes; random tokens do not.
`token_digest` is unique and indexed	Lookup by digest is the hot path on every authenticated request.

Expiry and revocation live in the token row: `expires_at` bounds the session, and `revoked_at` lets logout kill it before it expires. `resolve()` rejects both cases, so the check is centralized rather than repeated per route.

> **Trade-off to notice:** opaque tokens require a database lookup per request, but allow instant revocation. A stateless JWT avoids that lookup but cannot be revoked without extra state. EDMS chose revocation simplicity.

Never log tokens. A token in a log file is a working credential. The same applies to the raw `Authorization` header.

17. Time, expiry, and naive versus aware datetimes

```
def utc_now() -> datetime:
    return datetime.now(timezone.utc).replace(tzinfo=None)
```

Python distinguishes **naive** datetimes (no timezone) from **aware** ones (carry `tzinfo`). Comparing or subtracting a naive and an aware datetime raises `TypeError`.

- The helper computes the current UTC time and strips `tzinfo`, so everything stored and compared follows one convention.
- The columns are `DateTime` without timezone support. Mixing conventions is a classic source of off-by-hours bugs and runtime errors.
- `expires_at = utc_now() + ttl` is computed once, at token creation; later checks compare against a fresh `utc_now()`.
- **Server time is authoritative.** The client never decides whether a token is still valid.

If the project ever needs multiple timezones or DST-aware reporting, switch to timezone-aware columns and convert only at the presentation boundary — do it deliberately, not by mixing.

18. Custom exceptions: structure, messages, and chaining

An exception class is a *type* that callers can catch precisely. It needs no methods to work; the class body only needs at least one statement. A docstring satisfies that, which is why replacing `pass` with a docstring changed nothing about behavior.

```
class InvalidTokenError(Exception):
    """Raised when a bearer token is unknown, expired, or revoked."""

    def __init__(self, reason: str = "Token is invalid or expired.") -> None:
        super().__init__(reason)
```

- **`super().__init__(message)`** sets `args`, so `str(exc)` and traceback printing show a useful message. Without it, the exception prints with no detail.
- **Extra attributes** (`self.username`, `self.permission`, `self.value`) let upper layers respond programmatically instead of parsing text.
- **Catch by type**: `except InvalidCredentialsError` does not accidentally swallow unrelated `ValueError`s.
- **Raise with a cause**: `raise CategoryAlreadyExistsError(name) from error` preserves the original `IntegrityError` as `__cause__`, so the traceback shows both.

```
try:
    session.flush()
except IntegrityError as error:
    session.rollback()
    raise CategoryAlreadyExistsError(category.name) from error
```

This converts a database-specific failure into a domain-specific one at the repository boundary. The API layer then maps that single type to HTTP 409 without knowing SQLAlchemy exists.

19. The bootstrap pattern and the TOCTOU race

```
def bootstrap_admin(self, username, email, password) -> User:
    if self.user_repository.count() > 0:
        raise PermissionDeniedError("auth:manage")
    return self.register_user(username, email, password, Role.ADMINISTRATOR)
```

Registration requires `auth:manage`, which nobody holds until an administrator exists. The bootstrap endpoint breaks that circular dependency by allowing exactly one unauthenticated administrator creation.

This is a **TOCTOU** check ("time of check to time of use"): two simultaneous bootstrap requests could both read `count() == 0` before either inserts. The check narrows the window; it does not close it.

- Real protection comes from the database: a uniqueness rule (for example a singleton row or unique username) plus a transaction. The second writer fails and rolls back.
- The `403` here means "bootstrap already used", not "you are not logged in" — a deliberate distinction worth documenting in the API.
- In production, bootstrap is usually a one-off CLI task or an environment-gated endpoint, not an open public route.

20. FastAPI fundamentals: ASGI, app factory, routers

FastAPI runs on ASGI, the asynchronous server interface implemented by Uvicorn. A request travels: server → middleware → router → dependencies → endpoint → response model → server.

```
def create_app(database_url: str = "sqlite:///edms.db") -> FastAPI:
    database = Database(database_url)
    database.create_schema()

    app = FastAPI(title="EDMS API", version="1.0.0")
    app.state.database = database

    for module in (auth, catalog, inventory, sales, procurement, logistics, reporting):
        app.include_router(module.router, prefix="/api")

    return app
```

Concept	Meaning in EDMS

App factory	`create_app(url)` builds a configured app. Tests pass a temporary database URL instead of touching the real file.
Router	One module per business area with `prefix` and `tags`, so paths and docs stay grouped.
`app.state`	Attach shared, long-lived objects (the `Database`) to the app instance; routes reach them through `request.app.state`.
OpenAPI	FastAPI derives a machine-readable schema from routes and Pydantic models; `/docs` renders it.

Creating the schema inside `create_app` makes startup explicit and keeps tests self-contained. A production deployment would normally run migrations instead.

21. Dependency injection and the per-request session

`Depends` declares that an endpoint needs something; FastAPI builds it, in the right order, before calling the function.

```
def get_database(request: Request) -> Database:
    return request.app.state.database

def get_session(database: Database = Depends(get_database)):
    with database.session_scope() as session:
        yield session
```

- **Generator dependencies** use `yield` to separate setup from teardown. Code before `yield` runs before the endpoint; the block after it runs when the request finishes.
- **One session per request**, shared by the auth service and the route: FastAPI caches dependency results within a request by default, so `get_session` is not called twice.
- **The request is one transaction.** `session_scope()` commits on normal exit. If the endpoint raises, the exception passes through the context manager, which rolls back — so a half-finished sales order never commits.
- **`expire_on_commit=False`** keeps already-loaded attributes usable after commit instead of marking them stale and forcing a refresh query.
- **Teardown order matters.** Response values are built while the request is still active. After teardown the session closes; touching an unloaded lazy attribute later raises `DetachedInstanceError`. That is why routers convert domain objects into Pydantic responses inside the endpoint.

```
def require_permission(permission: str):
```

```
    def checker(user: User = Depends(get_current_user)) -> User:
        if not user.has_permission(permission):
            raise HTTPException(status_code=403, detail=...)
        return user
    return checker

# used as
current_user: User = Depends(require_permission("sales:write"))
```

This is a **dependency factory**: calling `require_permission(...)` returns a new callable that captures the permission. The returned dependency is what `Depends` receives.

> **Testing hook:** `app.dependency_overrides[get_session] = fake` can swap a real dependency for a test double. EDMS's API tests instead build the app with a temporary SQLite file, which exercises the real stack.

22. Pydantic models: the validation boundary

```
class SalesLineIn(BaseModel):
    configuration_id: int
    quantity: int
    unit_price: Decimal | None = None

class SalesOrderCreate(BaseModel):
    customer: CustomerIn
    warehouse_id: str
    lines: list[SalesLineIn]
```

- **Request models validate and coerce.** JSON strings such as `"5.00"` become `Decimal`; invalid shapes produce `422` before any business code runs.
- **Nested models** compose: `SalesOrderCreate` contains `CustomerIn` and a list of `SalesLineIn`.
- **`Field(default_factory=list)`** creates a fresh list per model instead of sharing one mutable default.
- **`model_dump()`** turns a model into a plain dictionary for code that expects dicts, such as the sales service's line input.
- **Response models filter.** `UserResponse` lists only public fields, so `password_hash` can never leak even if someone adds it to the returned object.
- **Decimal precision.** Money is kept as `Decimal`; JSON serialization typically emits it as a string to preserve exactness, so clients should parse rather than assume a float.

Two different failure modes are worth separating: a **422** means the request body did not match the schema; a **400** means the body was well formed but the business rule rejected it (for example, a

configuration ID that does not exist).

23. Mapping domain errors to HTTP without polluting the domain

```
ERROR_STATUS_CODES = {
    InvalidCredentialsError: 401,
    InvalidTokenError: 401,
    InactiveUserError: 403,
    PermissionDeniedError: 403,
    UsernameAlreadyExistsError: 409,
    CategoryAlreadyExistsError: 409,
    ValueError: 400,
}

def _error_handler(status_code: int):
    async def handler(request: Request, exc: Exception) -> JSONResponse:
        return JSONResponse(status_code=status_code, content={"detail": str(exc)})
    return handler

for error_type, status_code in ERROR_STATUS_CODES.items():
    app.add_exception_handler(error_type, _error_handler(status_code))
```

- **Services and repositories stay HTTP-free.** They raise `ValueError` or a domain exception. Only the app factory knows status codes.
- **Subclass matching.** When an exception is raised, the framework searches the exception's class hierarchy for a registered handler, so specific types win when both a base and a subclass are registered.
- **The factory avoids a classic bug.** Inside `_error_handler(status_code)`, each handler closes over its *own* parameter. If the handler were defined directly in the loop, every closure would share the loop variable and all responses would use the final status code — Python closures capture variables, not values.
- **Trade-off:** mapping the broad `ValueError` to 400 is convenient but can disguise genuine bugs as client errors. A dedicated validation-exception base class becomes worthwhile as the project grows.

24. HTTP semantics used by the API

Code	When EDMS uses it	Meaning
200	Successful reads and lifecycle actions	Request succeeded with a body
201	Creating a category, product, order, lot, user	A new resource exists

204	Logout	Success, deliberately no body
400	Unknown configuration ID, invalid lifecycle transition	Well-formed request, rule violation
401	Missing/malformed header, bad token, wrong password	Identity not established
403	Insufficient permission, inactive user	Identity known, action refused
404	Unknown product, order, shipment	Resource not found
409	Duplicate username, category, or specification	Conflicts with current state
422	Body fails Pydantic validation	Schema mismatch (framework-level)

```
def get_bearer_token(authorization: str | None = Header(default=None)) -> str:
    if not authorization or not authorization.lower().startswith("bearer "):
        raise HTTPException(
            status_code=401,
            detail="Missing or malformed Authorization header.",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return authorization.split(" ", 1)[1].strip()
```

- **Scheme check is case-insensitive**, as the HTTP specification requires.
- **`split(" ", 1)`** splits only on the first space, so a token that contained a space would not be silently truncated at the wrong place.
- **`WWW-Authenticate: Bearer`** tells compliant clients which scheme to use.
- **POST actions versus PATCH state.** `/orders/{id}/pay` names a business operation; `PATCH /orders/{id}/status` sets a field. Naming operations as endpoints keeps lifecycle rules server-side instead of trusting a client-supplied status.

25. Testing an API in-process

```
self.tmpdir = tempfile.mkdtemp()
database_url = f"sqlite:///{os.path.join(self.tmpdir, 'test.db')}"
self.app = create_app(database_url)
self.client = TestClient(self.app)
```

- **No network.** `TestClient` calls the ASGI app directly — real routing, dependency injection, validation, and exception handlers, without starting Uvicorn.
- **File, not `sqlite://`.** An in-memory SQLite database lives inside one connection; a pool that opens a second connection sees an empty database with no tables. A temporary file gives every connection the same data.
- **Isolation per test.** A fresh directory per test method means no ordering dependencies. `tearDown` disposes the engine and removes the file.
- **Assert on status and body.** Permission tests assert 403 for a forbidden write and 200 for an allowed read — proving both halves of the rule.
- **Warnings are not failures.** The environment prints a `StarletteDeprecationWarning` about `httpx`; it comes from the installed Starlette version, not from EDMS code.

Testing through HTTP catches wiring problems that unit tests miss: a missing schema model, a router included with the wrong prefix, a domain error that never maps to a status code.

26. Practice: API and authentication exercises

1. Create two users with the identical password and compare their stored hashes. Explain the difference using the salt.
2. Call `/api/auth/me` with no header, with `Basic`, and with a revoked token. Record the status codes and explain each.
3. Log in, then deactivate the user in the database and reuse the token. Which check rejects it, and with which code?
4. Add a new role to `ROLE_PERMISSIONS` and verify a forbidden route becomes allowed without touching any router.
5. Remove `required` from a Pydantic field and observe the 422 response body. Then send a well-formed request that breaks a business rule and compare the 400.
6. Make an endpoint raise `ValueError` after a successful `flush()`. Confirm the transaction rolled back by checking the database.
7. Predict, before running, why the handler factory in section 23 would break if it were written inline in the loop.
8. Write an API test that proves a sales officer cannot approve a purchase order, then explain which dependency produced the 403.

27. Source map

- `procurement/models/purchase_order.py`: domain behavior, lifecycle, totals, duplicate validation.
- `procurement/models/purchase_line.py`: quantity/price validation and receipt behavior.
- `procurement/repositories/purchase_order_repository.py`: ORM mapping, constraints, query construction, domain-to-record mapping.
- `procurement/services/purchase_line_service.py`: service delegation to the order repository.
- `shared/database.py`: engine, session factory, schema creation, and transaction context.
- `authentication/security.py`: PBKDF2 hashing, token generation, and token digests.
- `authentication/models/user.py`: roles, permission constants, role mapping, and the `User` model.
- `authentication/repositories/user_repository.py`: user and token ORM records, expiry/revocation lookup, and unique constraints.
- `authentication/services/authentication_service.py`: registration, bootstrap, login, token authentication, and permission checks.
- `authentication/exceptions.py`: typed errors that carry structured context.
- `API/main.py`: app factory, router registration, and domain-error-to-HTTP mapping.
- `API/dependencies.py`: per-request session, auth service, bearer parsing, current user, and permission dependency factory.
- `API/schemas.py`: Pydantic request/response models.
- `API/routers/*.py`: thin HTTP endpoints per business area.
- `tests/purchase_line_tests.py`: behavior and database-constraint examples.
- `tests/api_tests.py`: authentication, authorization, status-code mapping, and end-to-end HTTP flows.

Built as a learning aid for the EDMS catalog, inventory, procurement, sales, logistics, reporting, authentication, and API slices. Database schemas and framework APIs evolve; use migrations for changes to existing databases, and verify behavior against the actual database engine and dependency versions used in deployment.